How Large Language Models Are Built and Used

Scale, Post-Training, Chat Templates, Decoding and Limits

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

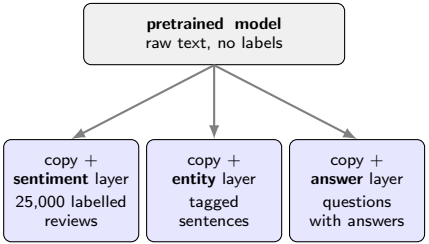
KAUST Academy
King Abdullah University of Science and Technology

November 10, 2026

Table of Contents

1. From a decoder to a large language model: tasks written as text, examples in the prompt, a base model and a chat model
2. Scale: model sizes and training tokens, compute by hand, scaling laws, compute-optimal and over-trained models
3. Pretraining data: from web crawl to clean tokens, cleaning rules by hand, quality classifiers and duplicates
4. From a base model to an assistant: instruction-tuning data and loss, then alignment with people's preferences
5. Chat templates: roles, special tokens, and what a wrong template does
6. Decoding: greedy and beam search, temperature, top-k and top-p, settings per task
7. The context window: its size, the cost of filling it, and what gets lost in a long input
8. Hallucination: why models make things up, measured, and what helps
9. Open and closed models, and Arabic-centric LLMs
10. Summary and references

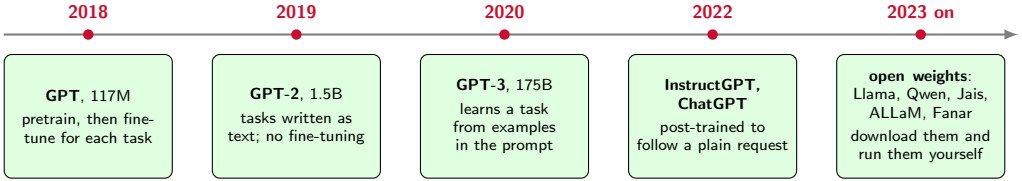
Before LLMs: One Fine-Tuned Model per Task



one training run and one stored model per task

- The usual recipe for encoders (BERT), decoders (GPT) and encoder-decoders (T5) alike: pretrain once on raw text, then fine-tune a copy for each task, with a new output layer and labelled examples. DistilBERT fine-tuned on 25,000 IMDb reviews reached 91.3%.
- **The problem:** every new task needs its own labelled data, its own training run, and its own copy of the model to store and serve.
- A pretrained decoder already writes text: after "The capital of Saudi Arabia is" it continues "Riyadh. The capital of the United States is Washington, D". **The question:** can the task itself be written as text, so that one model does many tasks with no training?

Large Language Models: From GPT to ChatGPT



- A **large language model** (LLM) is a decoder pretrained on next-token prediction at large scale, billions of parameters and trillions of tokens, and usually post-trained into an assistant.
- Each step removed a cost of the one before: first the training run for each task, then most of the labelled data, then the need to write every request in a special format.
- "Large" keeps moving. GPT-2's 1.5B was large in 2019. Qwen3-0.6B is called a small model, yet it read over 30T tokens: a hundred times the text GPT-3 read.

Radford et al., 2018 and 2019; Brown et al., 2020; Ouyang et al., 2022. GPT's size: the smallest GPT-2 model, "equivalent to the original GPT".

Write the Task as Text the Web Already Contains

the request, written as	Qwen3-1.7B-Base continues
Translate into Arabic: The library opens at nine in the morning.	"The library opens at nine in the morning."
`English:` The library opens at nine in the morning. `Arabic:`	المكتبة تفتح في الساعة التاسعة صباحاً. English: The library opens at nine in the morning. . . .
What is the capital of Saudi Arabia?	"Also, can you tell me about the largest desert in the world, . . ."
`Q:` What is the capital of Saudi Arabia? `A:`	Riyadh Q: What is the capital of the United States? A: Washington, D.C. . . .

The base model (pretraining only), most probable token at every step, 40 new tokens. Grey: what it wrote after the answer. Computed for this course.

- **The idea of GPT-2** (2019): web pages already contain tasks: questions followed by answers, sentences next to their translations, articles followed by a short summary. Write your task in that shape, and the most probable continuation is the answer.
- GPT-2, a 1.5B decoder trained on 40 GB of web pages, was never fine-tuned: "To induce summarization behavior we add the text TL;DR: after the article". This is **zero-shot**: no examples, no training.
- The limits: you must guess a format for every task, the model keeps writing after the answer, and quality is low. GPT-2's summaries "just barely" beat "selecting 3 random sentences from the article".

Radford et al., "Language Models are Unsupervised Multitask Learners," 2019, Sections 2 and 3.6.

GPT-3: Learn the Task From Examples in the Prompt

	what the model receives	weights change
zero-shot	`Translate English to French:` `cheese =>`	no

one-shot	`Translate English to French:`
	`sea otter => loutre de mer`
	`cheese =>`
		no

few-shot	`Translate English to French:`
	`sea otter => loutre de mer`
	`peppermint => menthe poivrée`
	`plush girafe => girafe peluche`
	`cheese =>`
		no

fine-tuning	`sea otter => loutre de mer`
	gradient update
	`peppermint => menthe poivrée`
	gradient update ... then
	`cheese =>`
		yes

Redrawn from Brown et al., Figure 2.1. The model writes the French after `cheese =>`.

- **The problem:** a format alone often does not say what is wanted. **The fix** in GPT-3 (175B, 2020): put a few solved examples before the real input.
- **Zero-shot**: the task description only. **One-shot**: one example. **Few-shot**: "a few demonstrations of the task at inference time as conditioning", 10 to 100 for GPT-3.
- "Without any gradient updates or fine-tuning": this is **in-context learning**. The examples are part of the input, read again with every request, and forgotten after it.
- Fine-tuning, as for DistilBERT, changes the weights with every batch of examples.

Brown et al., "Language Models are Few-Shot Learners," NeurIPS 2020, arXiv:2005.14165v4.

A Pretrained Decoder as a Classifier, Measured

model	size	format only	+ one-line instruction	+ 4 examples
GPT-2	124M	83.5%	74.0%	58.8%
Qwen3-0.6B-Base	0.6B	88.5%	90.5%	90.7%
Qwen3-1.7B-Base	1.7B	87.5%	93.0%	92.8%
DistilBERT, fine-tuned	66M	91.3%, after training on 25,000 labelled reviews		

200 IMDb test reviews (100 positive, 100 negative), first 200 words. Answer: the larger next-token probability of "positive" and "negative". Examples: 2 positive, 2 negative short training reviews; mean of 3 example sets (Qwen3-0.6B-Base: 86.0 to 93.5%). DistilBERT: all 25,000 test reviews. Computed for this course.

```
Classify each movie review as positive or
negative.
Review:  Good old black and white Graham Greene
...
Sentiment:  positive
Review:  ...  should never be watched unless
...
Sentiment:  negative
... two more examples ...
Review:  the review to classify
Sentiment:  p(positive) or p(negative)?
```

- No training and no new layer: one forward pass, then compare two probabilities.
- Qwen3-1.7B-Base matches the fine-tuned model from one instruction line. GPT-2 gets worse with both: "larger models make increasingly efficient use of in-context information" (Brown et al.).
- The price: 26 times DistilBERT's size, and 500 tokens per review with the examples.

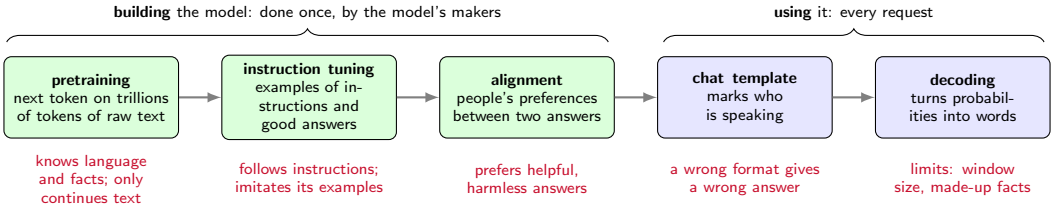
Base Model or Chat Model: The Same Request, Two Behaviours

request	base model, plain text	chat model, chat template
What is the capital of Saudi Arabia?	"Also, can you tell me about the largest desert in the world, the largest country in the world by land area, . . . "	"The capital of Saudi Arabia is Riyadh."
Translate into Arabic: The library opens at nine in the morning.	"The library opens at nine in the morning."	تُفتح المكتبة في الساعة التاسعة صباحًا.
Summarize in one sentence: The museum will close for renovation . . .	"The museum will reopen in September with a new exhibit, 'The Art of the American West,' . . . "	"The museum will close for renovation from June to August, with the gift shop and cafe remaining open, . . . "

Qwen3-1.7B-Base and Qwen3-1.7B: the same 1.7B-parameter decoder before and after post-training; most probable token at every step. Computed for this course.

- **The problem left:** patterns and examples work, but a plain request is still read as the start of a document. The base model continues it: more questions, the sentence again, an invented article.
- **The fix:** post-training. The same model after post-training, the chat model, treats the text as a request and answers. (The line is blurry: asked for an email to a manager, both wrote a good one.)

From Raw Text to an Assistant



- Each stage fixes what the one before leaves undone. Red: what the model can do, or what goes wrong, after that stage.
- The makers of a model choose the building stages; you choose the chat template and the decoding at every request. The first building question: how large a model, and how much text?

Learning Outcomes

By the end of this lecture you should be able to:

- **Use** a pretrained decoder for a task with no training, zero-shot and few-shot, and **explain** what post-training adds.
- **Estimate** the compute of a pretraining run, and **explain** scaling laws and why small models read far more text than is compute-optimal.
- **Describe** the stages from raw text to an assistant, **clean** web text with simple rules, and **pick** the right version of a model for an application.
- **Format** a conversation with a chat template, and **explain** what goes wrong without one.
- **Compute** greedy, temperature, top-k and top-p decoding by hand, and **choose** settings for a task.
- **Explain** the limits of the context window and why models hallucinate, and what reduces each.
- **Compare** open, closed and Arabic-centric models, and **choose** one for a requirement.
- **Prerequisite knowledge:** decoder-only transformers, next-token pretraining and fine-tuning; subword tokenization and the Arabic token premium; softmax and cross-entropy; PyTorch and Hugging Face `transformers`.

Bigger Models, Trained on Far More Text

model	year	size	training text	tokens/size
BERT-base	2018	110M	3.3B words	
GPT-3	2020	175B	300B tokens	1.7
Chinchilla	2022	70B	1.4T tokens	20
Llama 2	2023	70B	2T tokens	29
Llama 3	2024	405B	15.6T tokens	39
Qwen3	2025	0.6B	>30T tokens	>50,000

Size: parameters. M: million, B: billion, T: trillion. Sources: the models' papers (references).

- From BERT-base to Llama 3 405B, parameters grew about 3,700 times, and training text went from billions of words to trillions of tokens.
- Qwen3's tokenizer uses about 1.3 tokens per English word (measured on 1,000 English sentences), so 30T tokens is roughly 23 trillion words.
- **The problem:** more parameters and more text both cost compute. For a fixed budget, how large should the model be, and how much text should it read?

Compute by Hand: Six Operations per Parameter per Token

$$C \approx 6\,N\,D \qquad \underbrace{2N}_{\text{forward pass}} + \underbrace{4N}_{\text{backward pass}} \quad \text{floating-point operations per training token}$$

- N: parameters, D: training tokens. Each weight is used once per token, in one multiply and one add: $2N$. The backward pass costs about twice the forward pass.
- The rule matches what papers report, to within 1%.
- Generating one token needs only the forward pass: about $2N$ operations, 1.2 billion for Qwen3-0.6B.

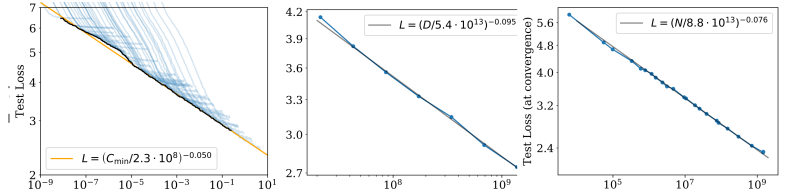
model	$6ND$	reported
GPT-3	3.15×10^{23}	3.14×10^{23}
Llama 3 405B	3.79×10^{25}	3.8×10^{25}
Qwen3-0.6B, 30T	1.1×10^{23}	

One T4 GPU peaks at 65 trillion operations per second:

$$\frac{1.1 \times 10^{23}}{6.5 \times 10^{13}\ /\text{s}} = 1.7 \times 10^{9}\ \text{s} \approx 53\ \text{years}$$

at full speed, for the smallest Qwen3.

Scaling Laws: Loss Falls as a Power Law



Test loss (log scale) against, left to right: compute (PF-days: petaflop/s for a day, 8.64×10^{19} operations), dataset size (tokens), parameters (excluding embeddings). Kaplan et al., "Scaling Laws for Neural Language Models," 2020, arXiv:2001.08361v1, Figure 1.

- "The loss scales as a power-law with model size, dataset size, and the amount of compute", with "some trends spanning more than seven orders of magnitude". On log-log axes, a power law is a straight line.
- For parameters, $L(N) = (N_c/N)^{0.076}$. By hand: doubling N multiplies the loss by $2^{-0.076} = 0.95$, and ten times N by $10^{-0.076} = 0.84$. Every tenfold step costs ten times more and buys the same 16% lower loss.
- The practical use: fit the line on small, cheap runs, and predict a large run before paying for it.
- **The question left open:** a budget of $C \approx 6ND$ can buy a larger model or more text. Which split gives the lowest loss?

Compute-Optimal: About 20 Tokens per Parameter

- Hoffmann et al. (2022) trained over 400 models, from 70 million to 16 billion parameters on 5 to 500 billion tokens, and found that "the model size and the number of training tokens should be scaled equally".
- They tested it with the compute of Gopher (280B parameters): Chinchilla, 70B parameters on "4× more data", 1.4T tokens, outperforms Gopher, GPT-3 and Megatron-Turing NLG (530B).
- Chinchilla's own ratio, 1.4T/70B, is **20 tokens per parameter**. GPT-3 had read 1.7 per parameter: earlier large models were trained on too little text for their size.

Sizing a run by hand. With $D = 20N$:

$$C = 6N \cdot 20N = 120\,N^2 \quad \Rightarrow \quad N = \sqrt{C/120}$$

For Qwen3-0.6B's budget, $C = 1.1 \times 10^{23}$:

$$N = \sqrt{9 \times 10^{20}} = 3 \times 10^{10}$$

a 30B-parameter model on 600B tokens.

Hoffmann et al., "Training Compute-Optimal Large Language Models," NeurIPS 2022, arXiv:2203.15556v1.

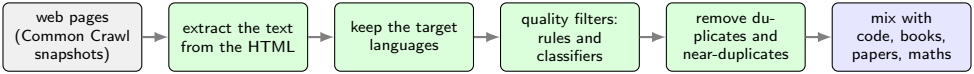
Why Small Models Are Trained Past 20 Tokens per Parameter

- **The problem:** 20 tokens per parameter minimizes the cost of **training**. A model is trained once, then answers millions of requests, and every generated token costs about $2N$ operations.
- **The fix:** spend the same training budget on a smaller model and much more text. Loss keeps falling with more data, slowly, along the same power law.
- Both models on the right cost the same to train. The small one is 50 times cheaper to run, and fits on one GPU.
- This is why current small models read tens of trillions of tokens: they are **over-trained** on purpose.

	compute-optimal	**Qwen3-0.6B**
parameters N	30B	0.6B
training tokens D	600B	>30T
tokens / parameter	20	>50,000
training operations	1.1×10^{23}	1.1×10^{23}
per new token	6×10^{10}	1.2×10^{9}
16-bit weights	60 GB	1.2 GB

Compute-optimal: the size that 20 tokens per parameter gives for the same budget, $C = 6N \cdot 20N$, so $N = \sqrt{C/120}$. Computed with 2 bytes per weight.

From Web Crawl to Trillions of Clean Tokens



- **The problem:** pretraining needs trillions of tokens, and only the web has that much text. Common Crawl is a public archive of web pages, collected in repeated snapshots; in 2019 one month gave "around 20TB of scraped text data". Much of a raw page is menus, buttons, cookie notices, spam, or a copy of another page.
- **The fix:** a pipeline of filters, run on every page. Each step throws text away; what is left decides what the model learns.
- Two public examples. **C4** (built for T5) cleaned one month of Common Crawl with simple rules into about 750 GB of English. **FineWeb** is "a 15-trillion token dataset derived from 96 Common Crawl snapshots"; its authors "document and ablate all of the design choices" by training a small model on each variant of the data.

Raffel et al., JMLR 2020, arXiv:1910.10683v4, Section 2.2; Penedo et al., "The FineWeb Datasets," 2024, arXiv:2406.17557v2.

Cleaning Rules by Hand: One Web Page

line of the page	rule
Home | News | Sports | Contact us	out: no end punctuation
Please enable Javascript to view this page.	out: “javascript”
The new metro line links the airport to the city centre in 25 minutes.	kept
Trains run every four minutes at peak times.	kept
Read more.	out: under 5 words
Tickets are sold at every station and in the mobile app.	kept
This site uses cookies; see our privacy policy.	out: “privacy policy”
© 2024 City News. All rights reserved.	kept: passes every rule

A made-up news page; outcomes checked with the code on the right.

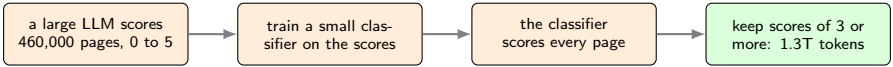
```
END = (".", "!", "?", '"')
BAD = ("javascript", "privacy policy",
       "terms of use", "uses cookies")

def keep_line(line):
    low = line.lower()
    return (line.rstrip().endswith(END)
            and len(line.split()) >= 5
            and not any(b in low
                        for b in BAD))
```

- C4 kept “lines that ended in a terminal punctuation mark” with at least 5 words, and dropped pages with “fewer than 3 sentences”, “lorem ipsum”, a curly bracket or a listed bad word.
- Rules are fast and easy to check, but crude: the copyright line survives.

Rules from Raffel et al., JMLR 2020, arXiv:1910.10683v4, Section 2.2.

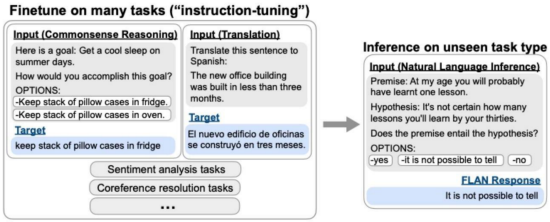
Quality Classifiers, Duplicates, and the Mixture



- **The problem:** rules catch broken lines, not useless pages; an advert in full sentences passes them all. **The fix:** a classifier that scores how useful a page is.
- **FineWeb-Edu**: Llama-3-70B-Instruct scored 460,000 pages "for their educational quality on a scale from 0 to 5"; a small model trained on 410,000 of those scores rated all of FineWeb, and "a minimum threshold of 3" kept 1.3 trillion tokens. Models trained on it did "dramatically better" on knowledge and reasoning benchmarks. The pattern is reusable: a large model labels a sample, a small model labels the rest.
- **Duplicates:** Lee et al. (2022) found one 61-word sentence repeated over 60,000 times in a standard corpus; deduplicated data trains models that "emit memorized text ten times less frequently".
- **The mixture** decides what the model is good at. Qwen3's 36T tokens cover 119 languages and dialects, with code, STEM (science, technology, engineering, maths) text, books and synthetic data.

Penedo et al., 2024, arXiv:2406.17557v2, Section 4; Lee et al., ACL 2022, arXiv:2107.06499v2; Qwen Team, "Qwen3 Technical Report," 2025, arXiv:2505.09388v1, Section 3.1.

Instruction Tuning: Instructions Paired with Good Answers



Wei et al., "Finetuned Language Models Are Zero-Shot Learners," ICLR 2022, arXiv:2109.01652v5, Figure 1 (top).

- **The problem:** a base model answers when its training text happened to contain answers, and drifts otherwise.
- **The fix:** fine-tune it on many tasks written as instructions, each with a good answer. This **supervised fine-tuning** (SFT) is also called instruction tuning.
- FLAN instruction-tuned a 137B model "on over 60 NLP tasks". On task types it never saw, it beat zero-shot 175B GPT-3 "on 20 of 25 tasks". Flan-PaLM later used 1.8K tasks.

What Instruction-Tuning Data Looks Like

```
{"messages": [
{"role": ..., "content": ...}, ...]}
user:
ما هي أكبر دول الشرق الأوسط من حيث مساحة؟
assistant:
المملكة العربية السعودية هي أكبر دول
الشرق الأوسط من حيث المساحة.
```

One Standard Arabic row of the Aya dataset, written by a person: "Which is the largest Middle Eastern country by area?", "Saudi Arabia ..."

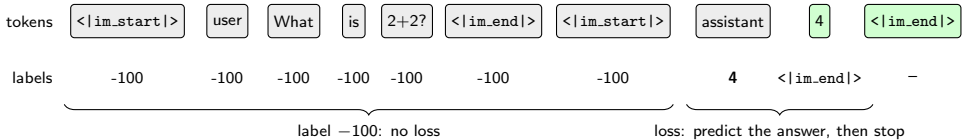
dataset	pairs	written by
No Robots	10,000	annotators, English
Aya	204,112	speakers of 65 languages

Aya training split: 4,995 Standard Arabic rows and 8,965 in Arabic dialects; counted for this course. Licences: Aya Apache 2.0; No Robots CC BY-NC 4.0, non-commercial.

- Each example is a short conversation: a list of messages with roles. Hugging Face TRL trains on it directly: "the trainer will automatically apply the chat template to the dataset", and `assistant_only_loss=True` puts the loss on the answers only.
- Answers are written by people, as here, or generated by a larger model: Qwen3's small models learn from the large Qwen3 models.
- LIMA fine-tuned a 65B model on "only 1,000 carefully curated prompts and responses": "almost all knowledge in large language models is learned during pretraining".
- **For your application:** instruction tuning teaches a format and a behaviour, not new facts. Facts belong in the prompt.

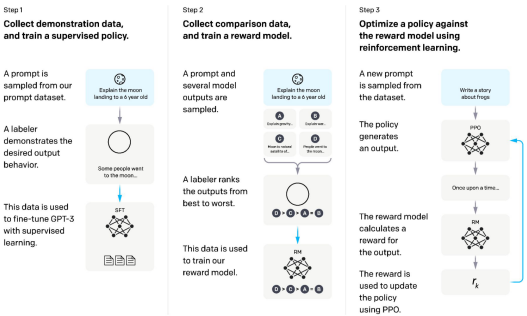
Singh et al., 2024; Zhou et al., NeurIPS 2023; Hugging Face TRL documentation (references).

One Training Example, and Where the Loss Goes



- The example is written in the chat format the model will be used with (tokens simplified here). The objective is the pretraining one, next-token cross-entropy.
- Each label is the token that should come next. Prompt positions get the label -100, which `nn.CrossEntropyLoss` skips by default: the model is not trained to write the user's question, only to answer it.
- Green: the answer. Training includes predicting the end token `<|im_end|>` after it; a model that never learned to stop keeps writing.

Alignment: People Choose the Better Answer



Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," NeurIPS 2022, arXiv:2203.02155v1, Figure 2.

- **The problem:** SFT copies its examples. It cannot learn that one answer is *better* than another, or what to avoid: harmful, invented or evasive answers.
- **The fix:** people compare answers to the same prompt, and the model is trained towards the kind they prefer. **RLHF** (reinforcement learning from human feedback) trains a scorer on their choices, then trains the model to please it; **DPO** (direct preference optimization) trains on the pairs directly.
- It works: people preferred the answers of the 1.3B InstructGPT model to those of the 175B GPT-3.
- **For your application:** use the chat version (Qwen3-0.6B, not Qwen3-0.6B-Base). To adapt it, start with SFT on your examples; preference tuning needs only pairs of a better and a worse answer, and libraries such as Hugging Face TRL run it.

One Stream of Tokens: the Template Marks Who Speaks

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Capital of Saudi Arabia?<|im_end|>
<|im_start|>assistant
Riyadh.<|im_end|>
<|im_start|>user
And of Qatar?<|im_end|>
<|im_start|>assistant
<think>

</think>
```

A three-turn conversation as Qwen3-0.6B receives it, printed by its tokenizer. The model writes from the end.

- **The problem:** a conversation has speakers and turns, but a decoder sees one sequence of tokens.
- **The fix:** special tokens, each one entry in the vocabulary, mark where a turn starts (`<|im_start|>` and a role) and ends (`<|im_end|>`). The model learns them during SFT.
- The **system** turn sets behaviour for the whole conversation. Earlier turns are sent again with every request: the model keeps no memory between calls.
- The prompt ends with the opening of an assistant turn, so the most likely continuation is the answer. The empty `<think>` block switches Qwen3's thinking off.

Chat Templates in Code

```
from transformers import (AutoTokenizer,
                          AutoModelForCausalLM)
name = "Qwen/Qwen3-0.6B"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name)

messages = [{"role": "user", "content":
             "What is the capital of Saudi Arabia?"}]
prompt = tok.apply_chat_template(
    messages, tokenize=False,
    add_generation_prompt=True,  # answer turn
    enable_thinking=False)       # Qwen3 only
ids = tok(prompt, return_tensors="pt")
out = model.generate(**ids, max_new_tokens=40,
                     do_sample=False)
answer = out[0, ids["input_ids"].shape[1]:]
print(tok.decode(answer, skip_special_tokens=True))
# The capital of Saudi Arabia is **Riyadh**.
```

- The template ships with the tokenizer, as a small program that turns the list of messages into text. Every model family has its own.
- `add_generation_prompt=True` appends the opening of the assistant turn; without it the model may continue the user's message.
- Slicing off the prompt's tokens keeps only what the model wrote. `**` is Markdown bold: chat models are trained to format their answers.

A Wrong Template Gives a Wrong Answer, Measured

prompt	**plain text**	**another model's format** `[INST] ... [/INST]`	**Qwen3's own template**
Give me three tips for learning Arabic.	"Also, if I have a problem with my phone, what should I do? What is the best way to learn Arabic? ..."	"The first tip is to practice daily. ... [END] # Instruction # Give me three tips ..."	"Sure! Here are three practical tips ... 1. Practice daily ..."
What is the capital of Saudi Arabia?	"... Riyadh. This is a correct statement? Yes No Question: What is the capital ..."	"A: The capital of Saudi Arabia is Riyadh. The question is asking ..."	"The capital of Saudi Arabia is **Riyadh**."

Qwen3-0.6B, most probable token at every step, 50 new tokens. Computed for this course.

- Plain text: the model continues the user's message, then turns it into a quiz. A foreign format: it answers, then keeps writing, imitating markup and commentary it half-recognises.
- Always use the tokenizer's own template. Libraries and local model servers apply it for you; a hand-built prompt string is where this goes wrong.

The Model Gives a Distribution; Decoding Picks the Token

“The capital of Saudi Arabia is”		“My favourite food is”	
Riyadh	0.251	pizza	0.055
__	0.040	a	0.044
____	0.038	chocolate	0.028
Riy	0.032	sushi	0.027
in	0.031	the	0.027
entropy	6.1 bits	entropy	8.9 bits

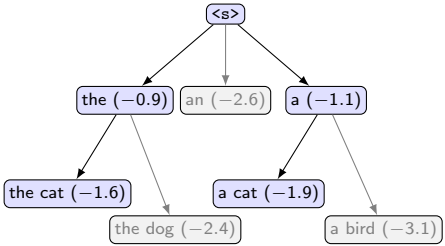
Next-token probabilities of Qwen3-0.6B-Base, top 5 of 151,669.

Computed for this course.

- At every step the model outputs one probability per vocabulary token. **The problem:** which token to write, and repeat 1,000 times without drifting?
- A factual prompt puts its largest share on one token. Even here, the base model gives about 13% to blanks such as __: it has read many fill-in-the-blank quizzes.
- An open prompt spreads the probability: **entropy**, $-\sum_i p_i \log_2 p_i$, measures the spread in bits.
- Decoding rules either take the most probable text, or sample from a reshaped distribution.

Greedy and Beam Search Choose the Most Probable Text

- **Greedy decoding** writes the most probable token at every step. A token that looks best now can force a worse continuation later, with no way back.
- **Beam search** keeps the k best partial outputs (the beam). Each step extends every one by every token, scores them by total log-probability, and keeps the top k. $k = 1$ is greedy decoding.
- Both suit tasks with one right answer: translation, extraction, a label, code that must compile.
- Both are **deterministic**: the same prompt always gives the same output.



Beam of $k = 2$ with illustrative total log-probabilities: only the two best partial outputs (blue) survive each step.

The Most Probable Text Repeats Itself, Measured

decoding	distinct 4-grams	loops
greedy	0.39	95%
beam search, $k = 4$	0.40	100%
sample, $T = 0.7$, top-p 0.8	0.85	50%
sample, $T = 1.0$, top-p 0.95	0.99	5%

Qwen3-0.6B-Base continuing the first 20 words of 20 IMDb reviews for 128 tokens. Distinct 4-grams: share of 4-token sequences that are not repeats. Loop: an 8-token sequence occurs twice. Computed for this course.

- **The problem:** the most probable continuation of open-ended text is not good text. Holtzman et al. found that "maximization-based decoding methods such as beam search lead to degeneration": text that is "bland, incoherent, or gets stuck in repetitive loops".
- Once a phrase has been written, writing it again becomes more probable: "a positive feedback loop", which greedy decoding follows every time.
- **The fix:** sample the next token from the distribution, after reshaping it, instead of always taking the top one.

Holtzman et al., ICLR 2020, arXiv:1904.09751v2.

Temperature Reshapes the Distribution

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \qquad z_i\text{: the logit (score) of token } i,\ T\text{: the temperature}$$

By hand: three tokens with logits $z = (2, 1, 0)$.

T	p_1	p_2	p_3
0.5	0.867	0.117	0.016
1	0.665	0.245	0.090
2	0.506	0.307	0.186

- $T = 1$: the model's own distribution. $T < 1$ sharpens it; $T \to 0$ is greedy decoding. $T > 1$ flattens it.

	$T = 0.5$	$T = 1$	$T = 1.5$
p("the")	0.84	0.49	0.14
p("that")	0.13	0.19	0.08
entropy (bits)	0.8	3.4	9.7
top-p 0.9 keeps	2	19	6,370

Qwen3-0.6B-Base, next token after "The best thing about living in Riyadh is". Computed for this course.

- At $T = 1.5$ thousands of tokens share the probability, most of them nonsense here: sampling from that tail is how text turns incoherent.

Three Samples at Three Temperatures

T	**"The best thing about living in Riyadh is" ...**
0.2	"... the beautiful and diverse city. The city is filled with a wide range of cultural and religious activities," "... the beautiful city and the beautiful people. I have been to many places in the world and I have"
0.7	"... the tight-knit community. The people are so friendly and welcoming. I found it difficult to make" "... the peaceful and quiet atmosphere, as there are fewer people around than in most cities. One of the"
1.5	"... Jeb Liquid South Beyond Fresh Frail walnut fi mple fakes clutch beyond" *[a Chinese character, a broken byte]* "... efficiently prioritize doing activity caputo collision driver lime bridge bc related"wcu proof with discretion"

Qwen3-0.6B-Base, 20 new tokens, two of three samples per temperature, full distribution. Computed for this course.

- Low temperature: safe, similar continuations. High: varied, then off topic, then broken. Sampling makes runs differ; fix the random seed to reproduce a result.

Top-k and Top-p Cut the Unreliable Tail

- **The problem:** even at a moderate temperature, thousands of unlikely tokens together hold real probability, and one unlucky draw derails the text.
- **Top-k:** keep the k most probable tokens, renormalize, sample.
- **Top-p** (nucleus sampling, Holtzman et al.): keep the smallest set of tokens whose probabilities add up to at least p, renormalize, sample.
- Top-p adapts to the model's confidence: at $T = 1$ and $p = 0.9$ it kept 19 tokens after "living in Riyadh is" and 1,003 after "My favourite food is". A fixed k cannot do both.

By hand: $p = (0.50,\ 0.20,\ 0.15,\ 0.10,\ 0.05)$, top-$p$ with $p = 0.8$.

Running sums: 0.50, 0.70, **0.85**: stop, keep 3 tokens.

Renormalize by 0.85:

$$(0.588,\ 0.235,\ 0.176)$$

Top-k with $k = 2$ would keep $(0.714,\ 0.286)$.

Hugging Face `generate` applies temperature, then top-k, then top-p; a token must pass every filter.

Decoding in Code, and Settings That Work

```
torch.manual_seed(0)
sampled = model.generate(
    **ids, max_new_tokens=200,
    do_sample=True, temperature=0.7,
    top_k=20, top_p=0.8)
greedy = model.generate(
    **ids, max_new_tokens=200,
    do_sample=False)
print(model.generation_config)
# do_sample: true, temperature: 0.6,
# top_k: 20, top_p: 0.95
```

The seed makes a sampled run reproducible. Without arguments, `generate` uses the model's `generation_config`: Qwen3-0.6B samples by default, with the values printed.

task	**decoding**
extraction, labels, translation, code	greedy, or a low temperature (0.3 or less)
chat (Qwen3, thinking off)	$T = 0.7$, top-p 0.8, top-k 20
reasoning (Qwen3, thinking on)	$T = 0.6$, top-p 0.95, top-k 20; "DO NOT use greedy decoding"
brainstorming, stories	higher T, top-p 0.9 to 0.95

Qwen3 rows: the Qwen3-0.6B model card, which also suggests a presence penalty (0 to 2) against endless repetition.

Every Model Has a Window, and the Answer Shares It

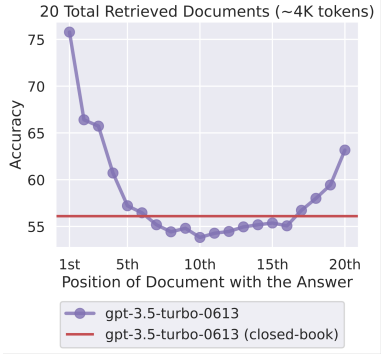
- The **context window** is the most tokens the model can attend to at once: system turn, earlier turns, documents, the question **and** the answer being written.
- Windows have grown: Llama 2, 4K tokens (2023); Llama 3, 128K (2024); Qwen3-0.6B, 32,768.
- Text beyond it is cut, usually the oldest turns, or the request fails. A chat that runs long silently forgets its beginning.
- Filling the window costs: attention scores grow with the square of the length, and the keys and values of every past token must be stored while writing.

Qwen3	tokens / word	words in 32,768
English	1.30	about 25,000
Arabic	2.22	about 14,800

Tokens per word measured on 1,000 English–Arabic sentence pairs from OPUS-100. Computed for this course.

The same window holds about 40% fewer Arabic words: the token premium of the tokenizer becomes a premium on memory, cost and how much of a document fits.

Long Inputs: Information in the Middle Is Used Least



Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," TACL, arXiv:2307.03172v3, Figure 1.

- **The problem:** fitting in the window does not mean being used. Liu et al. gave a model a question and 20 retrieved documents, about 4K tokens, one of which holds the answer, and moved that document from first to last.
- Accuracy "is often highest when relevant information occurs at the beginning or end of the input context", and drops in the middle: here, below the red line, the model answering with no documents at all.
- The drop held for open and closed models, "even for explicitly long-context models".
- Practical rules: send only the passages that matter; put the instructions and the question at the end, after the documents; rank the most relevant passages first.

Finding One Fact in a Long Text, Measured

input length	position of the asked-for passcode among the 50 1st	13th	26th	38th	50th (last)
2,000 tokens	100%	19%	31%	56%	38%
6,000 tokens	100%	19%	13%	44%	88%

Share of correct answers, Qwen3-0.6B, thinking off, greedy. 50 sentences "The passcode for the *X* archive is *nnnn*." spread through IMDb reviews; the question names one archive. 16 trials per cell. Computed for this course.

- With a single passcode hidden in the text, the same model found it in every trial, at every depth, in inputs of up to 8,000 tokens.
- With 50 similar sentences it must pick the right one. It always gets the first; in the middle it is right only 13 to 31% of the time. At 6,000 tokens the curve is the U of Liu et al.: 100% first, 88% last.
- Long-window claims are usually measured with the single-needle test. Measure on inputs like your own before relying on a long window.

Hallucination: Fluent, Confident and Wrong

- A **hallucination** is generated text that reads well and is false or unsupported: an invented fact, quote, citation or number.
- Why it happens:
  - pretraining rewards a *plausible* next token; nothing in the loss rewards saying "I don't know";
  - a fact seen a few times in 30T tokens is stored weakly, if at all;
  - the model knows nothing after its training data was collected;
  - sampling can pick a low-probability token, and the text then builds on it.

made-up question	**Qwen3-0.6B's answer**
Who won the 2019 Zarvonian Open chess tournament?	"The 2019 Zarvonian Open chess tournament was won by **Elena Kovalchuk**."
Who wrote the novel 'The Glass Orchards of Meridan'?	"... was written by **E. B. White**. It was published in 1917 and is a classic example of American literature."
In what year was the Tavrinsk Bridge completed?	"The Tavrinsk Bridge was completed in **1912**."

The tournament, the novel and the bridge are invented for this test: no right answer exists. Greedy decoding, thinking off. Computed for this course.

Allowing "I Don't Know" Helps, at a Price, Measured

prompt	20 made-up invented an answer	20 real questions correct	20 real questions abstained
the question alone	18	17	1
the question, plus "If you do not know the answer, say *I don't know*."	0	1	19

Qwen3-0.6B, greedy, thinking off. 20 questions about invented things, 20 easy real ones (capital of Egypt, chemical symbol for gold, ...). Abstention: the answer says it does not know, or that the thing does not exist (matched by phrases). Computed for this course.

- **The fix to try first:** make abstaining an allowed answer. It removed every invented answer.
- The price: the small model now says "I don't know" to 19 of 20 easy questions it had answered correctly. It cannot tell what it knows; it follows the instruction everywhere.
- Lowering the temperature does not add knowledge: greedy decoding was used throughout. Test such an instruction on questions with known answers before adopting it.

Larger Models Are Not Automatically More Truthful



Lin, Hilton and Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," ACL 2022, arXiv:2109.07958v2, Figure 2.

- TruthfulQA: 817 questions in 38 categories where people often answer falsely (myths, misconceptions). "The best model was truthful on 58% of questions, while human performance was 94%".
- "The largest models were generally the least truthful" (top), while on ordinary trivia larger models did better (bottom): a bigger model imitates human text, falsehoods included, more faithfully.
- What reduces hallucination in practice: put the source documents in the prompt and ask for answers from them, with quotes; allow abstention; check facts with a tool; and alignment that rewards honest answers.

Open-Weight and Closed Models

	open weights (Qwen3, Llama, Gemma, Mistral, Jais, ALLaM, Fanar)	**closed, behind an API** (GPT, Claude, Gemini)
access	download the weights, run them anywhere	send requests to the provider
data	never leaves your machines	sent to the provider
cost	hardware and electricity, whatever the traffic	a price per token, nothing up front
customization	fine-tune every weight, change decoding, inspect probabilities	the provider's options only
stability	the model never changes unless you change it	versions are updated and retired by the provider
quality	strong at small and middle sizes	the strongest models are mostly here

- "Open" usually means open *weights*: the training data and code are rarely published. Each model has a licence; Qwen3 uses Apache 2.0, others add conditions of their own.
- A common pattern: prototype with a closed API, then move the volume, or the sensitive data, to an open model you run yourself.

Arabic-Centric LLMs: Jais, ALLaM, Fanar

model	built by	parameters	how it was trained
Jais (2023)	Inception, MBZUAI and Cerebras (UAE, USA)	13B	from scratch on 395B tokens: Arabic repeated 1.6 times, English, code
ALLaM (2024)	National Center for AI, Saudi Data and AI Authority (SDAIA)	7B to 70B	7B from scratch: 4T English tokens, then 1.2T mixed Arabic and English; larger sizes start from Llama 2
Fanar (2025)	QCRI, Hamad Bin Khalifa University (Qatar)	7B, 9B	Fanar Star 7B from scratch on about 1T Arabic, English and code tokens; Fanar Prime 9B continued from Gemma-2 9B

- **The problem:** general models learn mostly from English, and the web holds far less Arabic. Three ways around the shortage: **repeat** it (Jais found 72B Arabic tokens and used them 1.6 times), **translate** into it (half of ALLaM's 540B curated Arabic tokens are translated from English), or **start from an English model** and continue pretraining on Arabic and English.
- All three were instruction-tuned into chat models, and all have open-weight versions on Hugging Face.

Sengupta et al., 2023, arXiv:2308.16149v2; Bari et al., 2024, arXiv:2407.15390v1; Fanar Team, 2025, arXiv:2501.13944v1.

Arabic-Centric Tokenizers Use Fewer Tokens, Measured

tokenizer	vocabulary	tokens per word		Ar / En
		English	Arabic	tokens
Llama 3.2	128,256	1.27	2.25	1.46
Qwen3	151,669	1.30	2.22	1.41
Fanar-1-9B	128,256	1.33	2.06	1.28
Jais (family)	84,992	1.29	1.44	0.92
ALLaM 7B	64,000	1.38	1.46	0.87

The same 1,000 English–Arabic sentence pairs from OPUS-100 (Zhang et al., 2020). Ar / En: Arabic tokens over English tokens for the same sentences. Computed for this course.

- Jais and ALLaM need about 1.45 tokens per Arabic word, against 2.2 for Qwen3 and Llama: the same Arabic text costs a third fewer tokens.
- They do it with smaller vocabularies, by spending them on Arabic words. On these pairs, Arabic even costs fewer tokens than English.
- Fewer tokens per word means more Arabic in the same window, faster generation and a lower bill per document. Fanar's tokenizer sits in between.

Summary

- **An LLM is a decoder pretrained at scale.** A task written as web-style text, or with a few examples in the prompt (in-context learning), needs no training: Qwen3-1.7B-Base sorted IMDb reviews 93% correctly. GPT-2 (124M) could not use instructions or examples.
- **A base model reads a plain request as a document;** after post-training, with its own chat template, the same model answers. Special tokens mark the turns; plain text or another model's format gives loops and wrong answers.
- **Compute is about** $6ND$**.** Loss falls as a power law; about 20 tokens per parameter minimizes training cost, but small models are over-trained on tens of trillions of tokens because every generated token costs $2N$.
- **Data is filtered web text plus code, books and maths:** line rules (C4), a quality classifier trained on a large model's scores (FineWeb-Edu), deduplication. Arabic is scarce: models repeat it, translate it, or continue from an English model.
- **Post-training turns a text continuer into an assistant:** instruction tuning on conversations with good answers (loss on the answer only; LIMA needed 1,000), then alignment on which of two answers people prefer (RLHF, DPO). Applications use the chat version.
- **Decoding:** greedy and beam search loop on open-ended text; sampling with temperature, top-k and top-p keeps it varied and on track. Use the settings on the model card.
- **Limits:** the window holds about 40% fewer Arabic than English words in Qwen3, and the middle of a long input is used least. Models invent answers to questions about things that do not exist; allowing "I don't know" and supplying sources help.
- **Choosing:** open weights for control and privacy, closed APIs for top quality without infrastructure; Arabic-centric models such as Jais, ALLaM and Fanar spend about a third fewer tokens on Arabic.

References

1. Radford, Narasimhan, Salimans and Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018.
2. Radford, Wu, Child, Luan, Amodei and Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, 2019. cdn.openai.com/better-language-models
3. Brown et al., "Language Models are Few-Shot Learners," NeurIPS 2020. arXiv:2005.14165v4
4. Maas et al., "Learning Word Vectors for Sentiment Analysis" (IMDb), ACL 2011.
5. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," NeurIPS 2022. arXiv:2203.02155v1 (source of Figure 2)
6. Kaplan et al., "Scaling Laws for Neural Language Models," 2020. arXiv:2001.08361v1 (source of Figure 1)
7. Hoffmann et al., "Training Compute-Optimal Large Language Models," NeurIPS 2022. arXiv:2203.15556v1
8. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. arXiv:2307.09288v2
9. Grattafiori et al., "The Llama 3 Herd of Models," 2024. arXiv:2407.21783v3

References (cont.)

10. Qwen Team, "Qwen3 Technical Report," 2025. arXiv:2505.09388v1; model card of Qwen3-0.6B, huggingface.co/Qwen/Qwen3-0.6B
11. NVIDIA, T4 Tensor Core GPU specifications. nvidia.com/en-us/data-center/tesla-t4
12. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (C4), JMLR 2020. arXiv:1910.10683v4
13. Penedo et al., "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale," 2024. arXiv:2406.17557v2
14. Lee et al., "Deduplicating Training Data Makes Language Models Better," ACL 2022. arXiv:2107.06499v2
15. Wei et al., "Finetuned Language Models Are Zero-Shot Learners," ICLR 2022. arXiv:2109.01652v5 (source of Figure 1)
16. Chung et al., "Scaling Instruction-Finetuned Language Models," 2022. arXiv:2210.11416v5
17. Singh et al., "Aya Dataset: An Open-Access Collection for Multilingual Instruction Tuning," 2024. arXiv:2402.06619v1; huggingface.co/datasets/CohereLabs/aya_dataset
18. Hugging Face H4, "No Robots" dataset. huggingface.co/datasets/HuggingFaceH4/no_robots

References (cont.)

19. Zhou et al., "LIMA: Less Is More for Alignment," NeurIPS 2023. arXiv:2305.11206v1
20. Hugging Face TRL documentation, "SFT Trainer." huggingface.co/docs/trl/sft_trainer
21. Holtzman, Buys, Du, Forbes and Choi, "The Curious Case of Neural Text Degeneration," ICLR 2020. arXiv:1904.09751v2
22. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," TACL. arXiv:2307.03172v3 (source of Figure 1)
23. Lin, Hilton and Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," ACL 2022. arXiv:2109.07958v2 (source of Figure 2)
24. Zhang et al., "Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation" (OPUS-100), ACL 2020. arXiv:2004.11867v1
25. Sengupta et al., "Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative Large Language Models," 2023. arXiv:2308.16149v2
26. Bari et al., "ALLaM: Large Language Models for Arabic and English," 2024. arXiv:2407.15390v1
27. Fanar Team, "Fanar: An Arabic-Centric Multimodal Generative AI Platform," 2025. arXiv:2501.13944v1